

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67\agent-a0d7936889e95cc11.jsonl`  
- SHA-256: `58c082f5664153dc6a2ac87b6e5f54fb53c152186e5918da51fe276cfccbc93`  
- Source modified: `2026-06-18T01:41:21+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_49c1018a-e67`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T01:40:32.123Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base  
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.
```

CANDIDATE:

```
{  
  "title": "Unguarded b[\"precision\"]/b[\"recall\"] on a persisted (committed, hand-editable) regression-baseline record",  
  "file": "backend/eval/regression.py",  
  "line": "129",  
  "trigger": "regress() loads the tracked eval_baselines/regression_baseline.json via load_baselines() (plain json.load, no validation) and looks up the record b = baselines.get(\"<video_id>::<stage>\"). If that file contains a baseline entry missing the \"precision\" or \"recall\" key ? e.g. a hand-edited entry, a partial/older-schema record, or one produced/merged outside save_baseline ? line 129 `base_p, base_r = b[\"precision\"]`, b[\"recall\"]` raises KeyError and aborts the regression run.",  
  "why_wrong": "b is a record deserialized from a persisted, explicitly TRACKED-and-committable JSON artifact (the module docstring/constant call it out as a checked-in file CI loads). Every OTHER field on the very same record is read defensively on the adjacent lines (line 132: b.get(\"iou_threshold\"), b.get(\"detector\"), b.get(\"gt_hash\")), and the function is clearly designed to degrade gracefully around baselines (has_baseline flag, the 'no baseline recorded' branch, the incommensurable-comparison reasons). Reading the two metric fields with bare [...] indexing is an inconsistency that turns a malformed/partial baseline entry into an uncaught KeyError crash instead of a handled condition.",  
  "severity": "low",  
  "proposed_fix": "Read the metric fields defensively like the sibling fields and treat a record lacking them as having no usable baseline. E.g.: `base_p, base_r = b.get(\"precision\"), b.get(\"recall\")` and guard the two regression comparisons with `if base_p is not None and s.precision < base_p - tolerance:` / `if base_r is not None and ...`; when either is missing, append a reason like 'baseline record missing precision/recall (malformed)' and skip the metric comparison (optionally set has_baseline=False) rather than KeyError.",  
  "confidence": 0.5,  
  "klass": "unguarded access on EXTERNAL or PERSISTED data"  
}
```

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/backend/eval/regression.py and related code/tests READ-ONLY before judging.

LENS ? REFUTE: try hard to show it is intended / already guarded / tested. Default real=false unless you can prove it is a genuine bug.

2. assistant (2026-06-18T01:40:35.633Z)

I'll analyze this candidate bug by reading the actual code in the worktree.

3. user (2026-06-18T01:40:36.339Z)

```
1      """Regression runner ? verify the production tool against golden-set ground truth.
2
3      Implements the explicit ask: *"given a video, get a golden set created and verify
4      its precision and recall against the production tool."* This is deliberately a thin
5      wrapper over machinery that already exists:
6
7      ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8      predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
9      comparison    ? interval matching + P/R/F1                  [backend.eval.metrics]
10     baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12     A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13     baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14     later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
39         different labels."""
40         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
41         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
```

```

56     reasons: List[str]
57     # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58     # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59     has_baseline: bool = False
60     gt_hash: Optional[str] = None
61
62     def as_dict(self) -> dict:
63         return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                      f1: float, path: str = DEFAULT_BASELINE_PATH,
82                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                      gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
detected
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100 _baseline_key(video_id, stage), precision, recall)
101
102     # ----- the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105               stage: str = "final", iou_threshold: float = 0.5,
106               detector: Optional[str] = None,
107               tolerance: float = DEFAULT_TOLERANCE,
108               baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
baseline.
110
111         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113         precision OR recall falls more than `tolerance` below the snapshotted baseline.
114         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115         caller can choose to snapshot the current numbers via `save_baseline`.
116         """

```

```

117     preds = get_predictions(db, video_id, stage, detector=detector)
118     s = metrics.score(preds, ground_truth, iou_threshold)
119     fp = gt_hash(ground_truth)
120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148     else:
149         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151     return RegressionResult(
152         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153         precision=s.precision, recall=s.recall, f1=s.f1,
154         baseline_precision=base_p, baseline_recall=base_r,
155         tolerance=tolerance, regressed=regressed, reasons=reasons,
156         has_baseline=has_baseline, gt_hash=fp,
157     )
158
159
160 def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                          tier: str = "file", stage: str = "final",
162                          iou_threshold: float = 0.5,
163                          detector: Optional[str] = None,
164                          tolerance: float = DEFAULT_TOLERANCE,
165                          baseline_path: str = DEFAULT_BASELINE_PATH,
166                          guideline: Optional[str] = None,
167                          sport: str = "badminton",
168                          provider_kwargs: Optional[dict] = None) -> RegressionResult:
169     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
once those providers land). `video_ref` is whatever that provider expects (a CSV

```

```

173     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174     predictions in the DB (the file MD5).
175     """
176     provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                   detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185               f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
188                   tol={r.tolerance}"
189             return head + ".join(f"\n    - {why}" for why in r.reasons)

```

4. assistant (2026-06-18T01:40:39.218Z)

Now let me look at the persisted baseline file and how it's produced/consumed, plus any tests and CLI callers, to judge whether a record missing "precision"/"recall" can realistically reach line 129.

5. user (2026-06-18T01:40:41.342Z)

FILE NOT FOUND

6. user (2026-06-18T01:40:42.145Z)

```

total 44
drwxr-xr-x 1 avidu 197609    0 Jun 18 06:56 .
drwxr-xr-x 1 avidu 197609    0 Jun 18 07:00 ..
-rw-r--r-- 1 avidu 197609 4696 Jun 18 06:56 experiment_leaderboard.json
drwxr-xr-x 1 avidu 197609    0 Jun 18 06:56 fixtures
-rw-r--r-- 1 avidu 197609 1977 Jun 18 06:56 gemini_wrapper_2026-06-10.json
-rw-r--r-- 1 avidu 197609  689 Jun 18 06:56 heuristic_lovo_n6.json
-rw-r--r-- 1 avidu 197609 9187 Jun 18 06:56 nightly_baseline.json

```

7. assistant (2026-06-18T01:40:45.013Z)

The committed `regression\_baseline.json` doesn't even exist. Let me check what tests and callers exist for `regress()`, and look at the fixtures dir and how baselines are loaded/written across the codebase.

8. user (2026-06-18T01:40:46.646Z)

```

.claude\worktrees\hunt-base\run_regression.py:62:     result = regression.regress_with_provider(
.claude\worktrees\hunt-base\run_regression.py:71:
regression.save_baseline(result.video_id, result.stage, result.precision,
.claude\worktrees\hunt-base\docs\CODE_MAP.md:98:         -> regression.regress(_with_provider) vs
baseline @backend/eval/regression.py [CI gate]
.claude\worktrees\hunt-base\docs\CODE_MAP.md:122:- `@run_regression.py::main` ?
`regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate),
2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run);
`::default_db_path` resolves via `backend.config.load_config`.
.claude\worktrees\hunt-base\docs\CODE_MAP.md:180:[Omitted long matching line]
.claude\worktrees\hunt-base\docs\BACKLOG.md:46:- ? **GS-5 ? regression triggers** ? wire
`regress()` into CI (pre-merge gate), on-demand CLI (the "surprise" case), and a scheduled run;
include a deliberate re-baseline path. **Revisit:** after GS-4.
.claude\worktrees\hunt-base\scripts\nightly_regression.py:509:def save_baseline(path: str,
snapshot: Dict[str, Any], tols: Tolerances) -> None:
.claude\worktrees\hunt-base\scripts\nightly_regression.py:567:

```

```

save_baseline(args.baseline, snapshot, tols)
.claude\worktrees\hunt-base\tests\test_automated_provider.py:89:    r =
regression.regress_with_provider(
.claude\worktrees\hunt-base\docs\archives\decisions\DECISIONS.md:274:4. **Regression = existing
harness + one new component.** `regress(video)` calls a provider for ground truth, runs the
production pipeline, and scores with the unchanged `metrics.py` against a per-video
`baseline.json`.
.claude\worktrees\hunt-base\docs\GOLDEN_SET_IMPLEMENTATION.md:22:
regress(video, tier) (GS-2) compares the two
.claude\worktrees\hunt-base\docs\GOLDEN_SET_IMPLEMENTATION.md:34:| **GS-2** | `feat/regression-
runner` | `regress(video, tier, tolerance)` over existing harness + `baseline.json`
snapshot/compare + CLI | GS-1 | No |
.claude\worktrees\hunt-base\docs\GOLDEN_SET_IMPLEMENTATION.md:52: `regress(video, tier="fast",
tolerance=0.05)` : get GT from `annotation_registry.get(tier)`, read predictions via the existing
`eval.harness`, score with the existing `metrics`, compare against a per-video `baseline.json`,
and flag a regression when precision/recall drop beyond tolerance. Pure orchestration over
existing parts + a small baseline store. CLI: `python run_regression.py --video ... --tier
file`.
.claude\worktrees\hunt-base\docs\GOLDEN_SET_IMPLEMENTATION.md:61:Wire `regress()` into CI (pre-
merge gate on pipeline changes), an on-demand CLI (the "surprise regression"), and an optional
scheduled run. Includes a deliberate re-snapshot path so legitimate production improvements can
rebaseline instead of raising false regressions.
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:77:- `regress(db, video_id, ground_truth,
stage="final", iou_threshold=0.5, detector=None, tolerance=DEFAULT_TOLERANCE,
baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ? score stored preds vs GT,
flag if P **or** R drops > tolerance.
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:78:- `regress_with_provider(...)` (:160)
? same, fetching GT via the annotation provider.
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:79:- `save_baseline(video_id, stage,
precision, recall, f1, ..., gt_fingerprint=None)` (:80) · `load_baselines(path) -> dict` (:68).
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:81:- Default baseline file:
`eval_baselines/regression_baseline.json` (:33).
.claude\worktrees\hunt-
base\docs\EXPERIMENT_HARNESS.md:111: `archives/research/AUDIO_RALLY_DETECTION_PLAN.md`. The
`regress()` + baseline path is the canonical comparison mechanism.
.claude\worktrees\hunt-base\docs\EXPERIMENT_HARNESS.md:153: `regression.regress()` +
`baseline.json` + `run_regression.py` exit codes. A variant becomes the new
.claude\worktrees\hunt-base\docs\GOLDEN_REGRESSION_FIXTURES.md:82:[Omitted long matching line]
.claude\worktrees\hunt-base\docs\GOLDEN_REGRESSION_FIXTURES.md:107:- **Quality-floor** ? score
vs committed labels; assert per-video + mean F1 ? floor in
`eval_baselines/regression_baseline.json` (created here; carries a **windowing fingerprint** so
a served-default change is flagged incommensurable, not silently compared); emit `eval_stats`
paired-delta-CI + sign test (NUMBERS INFORM); add a **metrics-vs-base-branch** paired delta.
Synthetic-tier floors measure **correctness, not field quality** (documented).
.claude\worktrees\hunt-base\docs\GOLDEN_REGRESSION_FIXTURES.md:156:| **M4** | **Quality-floor
gate + first `regression_baseline.json`. **`tests/test_golden_quality_floor.py`; create
`eval_baselines/regression_baseline.json` with **windowing fingerprint**; metrics-vs-base
paired-delta; corpus-level (not per-slug) LOVO for runtime. | M1, M2 | No | ? | ? |
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:8:[Omitted long matching line]
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:108: def regress(video, tier="fast",
tolerance=0.05):
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:121: **Triggers** (all feed the same
`regress()`): CI on pipeline changes · on-demand (the "surprise" case) · scheduled cron · pre-
release gate.
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:177:| 3 | `AutomatedProvider`
(independent oracle) + `regress()` + baseline snapshots | Phase 2 |
.claude\worktrees\hunt-base\docs\GOLDEN_SET_VENDORING.md:182:[Omitted long matching line]
.claude\worktrees\hunt-base\backend\eval\experiment.py:31: - the promotion gate stays
`run_regression.py` + `regression_baseline.json`
.claude\worktrees\hunt-base\backend\eval\experiment.py:508:    `regression_baseline.json`;
deliberately the size of a baseline file, not a service
.claude\worktrees\hunt-base\tests\test_eval_regression.py:1: """Tests for the regression runner
(GS-2): baseline store, regress(), the
.claude\worktrees\hunt-base\tests\test_eval_regression.py:27:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)

```

```

.claude\worktrees\hunt-base\tests\test_eval_regression.py:28:    b =
regression.load_baselines(p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:32:def
test_load_baselines_absent_is_empty(tmp_path):
.claude\worktrees\hunt-base\tests\test_eval_regression.py:33:    assert
regression.load_baselines(str(tmp_path / "nope.json")) == {}
.claude\worktrees\hunt-base\tests\test_eval_regression.py:36:# ----- regress() -----
.claude\worktrees\hunt-base\tests\test_eval_regression.py:39:    r = regression.regress(db,
"vid1", GT, baseline_path=str(tmp_path / "b.json"))
.claude\worktrees\hunt-base\tests\test_eval_regression.py:47:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:48:    r = regression.regress(db,
"vid1", GT, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:58:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:60:    r = regression.regress(d,
"vid1", GT, tolerance=0.05, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:66:def
test_within_tolerance_does_not_regress(tmp_path):
.claude\worktrees\hunt-base\tests\test_eval_regression.py:73:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:74:    r = regression.regress(d,
"vid1", GT, tolerance=0.05, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:80:def
test_regress_with_provider_file_tier(db, tmp_path):
.claude\worktrees\hunt-base\tests\test_eval_regression.py:83:    r =
regression.regress_with_provider(
.claude\worktrees\hunt-base\tests\test_eval_regression.py:115:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:147:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
.claude\worktrees\hunt-base\tests\test_eval_regression.py:150:    r = regression.regress(db,
"vid1", GT, iou_threshold=0.3, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_eval_regression.py:157:
regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
.claude\worktrees\hunt-base\tests\test_eval_regression.py:159:    r = regression.regress(db,
"vid1", [(0.0, 10.0), (21.0, 31.0)], iou_threshold=0.5, baseline_path=p)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:22:    load_baselines,
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:23:    save_baseline,
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:131:    """A user_id::stage
baseline record survives save ? load unchanged (sibling to regression_baseline.json)."""
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:135:    key = save_baseline(rec,
path=path)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:138:    loaded =
load_baselines(path)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:147:    save_baseline(rec2,
path=path)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:148:    both =
load_baselines(path)
.claude\worktrees\hunt-base\tests\test_per_user_regression.py:160:    saved =
load_baselines(path)
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:4:``regression_baseline.json``):
a candidate is gated so a global change can never silently
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:37:saved/loaded as JSON, a
sibling to `regression.py`'s ``regression_baseline.json`` approach.
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:62:    "load_baselines",
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:63:    "save_baseline",
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:137:def load_baselines(path: str
= DEFAULT_PER_USER_BASELINE_PATH) -> Dict[str, dict]:
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:145:def save_baseline(baseline:
PerUserBaseline,
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:149:    Round-trips through
`load_baselines`; returns the key written. Mirrors
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:150:
`regression.save_baseline` (atomic-ish read-modify-write of the one JSON file).

```

```
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:152:     baselines =
load_baselines(path)
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:280:     # per-user analogue of
regression.save_baseline. Does not change the decision above.
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:282:         save_baseline(
.claude\worktrees\hunt-base\backend\eval\regression.py:33:DEFAULT_BASELINE_PATH =
os.path.join("eval_baselines", "regression_baseline.json")
.claude\worktrees\hunt-base\backend\eval\regression.py:68:def load_baselines(path: str =
DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
.claude\worktrees\hunt-base\backend\eval\regression.py:80:def save_baseline(video_id: str,
stage: str, precision: float, recall: float,
.claude\worktrees\hunt-base\backend\eval\regression.py:90:     baselines = load_baselines(path)
.claude\worktrees\hunt-base\backend\eval\regression.py:104:def regress(db: Database, video_id:
str, ground_truth: List[metrics.Interval],
.claude\worktrees\hunt-base\backend\eval\regression.py:112:     AnnotationProvider (see
`regress_with_provider`). A regression is flagged when
.claude\worktrees\hunt-base\backend\eval\regression.py:115:     caller can choose to snapshot the
current numbers via `save_baseline`.
.claude\worktrees\hunt-base\backend\eval\regression.py:121:     baselines =
load_baselines(baseline_path)
.claude\worktrees\hunt-base\backend\eval\regression.py:160:def regress_with_provider(db:
Database, video_id: str, video_ref: str,
.claude\worktrees\hunt-base\backend\eval\regression.py:178:     return regress(db, video_id, gt,
stage=stage, iou_threshold=iou_threshold,
```

9. user (2026-06-18T01:40:47.384Z)

```
total 32
drwxr-xr-x 1 avidu 197609    0 Jun 18 06:56 .
drwxr-xr-x 1 avidu 197609    0 Jun 18 06:56 ..
-rw-r--r-- 1 avidu 197609 4221 Jun 18 06:56 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 18 06:56 clean_single_rally
-rw-r--r-- 1 avidu 197609 1557 Jun 18 06:56 manifest.json
drwxr-xr-x 1 avidu 197609    0 Jun 18 06:56 multi_rally_with_deadtime
drwxr-xr-x 1 avidu 197609    0 Jun 18 06:56 occlusion_break
```

10. assistant (2026-06-18T01:40:49.999Z)

Let me look at the test file to understand how baselines are exercised, and check whether any code path can write a baseline record lacking precision/recall (the only realistic non-hand-edit trigger).

11. user (2026-06-18T01:40:50.928Z)

```
1      """Tests for the regression runner (GS-2): baseline store, regress(), the
2      provider-driven end-to-end path, and the CLI exit-code contract."""
3      import pytest
4
5      from backend.infrastructure.database import Database
6      from backend.eval import regression
7      import run_regression
8
9
10     @pytest.fixture
11     def db(tmp_path):
12         d = Database(str(tmp_path / "t.db"))
13         d.add_video("vid1", "/x/m.mp4", "processed", [])
14         # Two confirmed segments matching the GT below ? perfect baseline.
15         d.add_play_segment("vid1", 0.0, 10.0)
16         d.add_play_segment("vid1", 20.0, 30.0)
17         return d
18
19
20     GT = [(0.0, 10.0), (20.0, 30.0)]
```



```

21
22
23 # ----- baseline store -----
24
25 def test_save_and_load_baseline_roundtrip(tmp_path):
26     p = str(tmp_path / "b.json")
27     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
28     b = regression.load_baselines(p)
29     assert b["vid1::final"]["precision"] == 1.0
30
31
32 def test_load_baselines_absent_is_empty(tmp_path):
33     assert regression.load_baselines(str(tmp_path / "nope.json")) == {}
34
35
36 # ----- regress() -----
37
38 def test_no_baseline_is_not_a_regression(db, tmp_path):
39     r = regression.regress(db, "vid1", GT, baseline_path=str(tmp_path / "b.json"))
40     assert r.regressed is False
41     assert r.precision == 1.0 and r.recall == 1.0
42     assert any("no baseline" in why for why in r.reasons)
43
44
45 def test_matching_baseline_passes(db, tmp_path):
46     p = str(tmp_path / "b.json")
47     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
48     r = regression.regress(db, "vid1", GT, baseline_path=p)
49     assert r.regressed is False
50
51
52 def test_recall_drop_beyond_tolerance_regresses(tmp_path):
53     # DB has only ONE of the two GT rallies ? recall 0.5 vs baseline 1.0.
54     d = Database(str(tmp_path / "t.db"))
55     d.add_video("vid1", "/x/m.mp4", "processed", [])
56     d.add_play_segment("vid1", 0.0, 10.0)
57     p = str(tmp_path / "b.json")
58     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
59
60     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
61     assert r.regressed is True
62     assert r.recall == pytest.approx(0.5)
63     assert any("recall" in why for why in r.reasons)
64
65
66 def test_within_tolerance_does_not_regress(tmp_path):
67     d = Database(str(tmp_path / "t.db"))
68     d.add_video("vid1", "/x/m.mp4", "processed", [])
69     d.add_play_segment("vid1", 0.0, 10.0)
70     d.add_play_segment("vid1", 20.0, 30.0)
71     p = str(tmp_path / "b.json")
72     # Baseline only 0.02 above current perfect score ? within default 0.05 tolerance.
73     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
74     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
75     assert r.regressed is False
76
77
78 # ----- provider-driven end-to-end (uses the GS-1 FileProvider) -----
79
80 def test_regress_with_provider_file_tier(db, tmp_path):
81     csv = tmp_path / "gt.csv"
82     csv.write_text("start,end\n0,10\n20,30\n")
83     r = regression.regress_with_provider(
84         db, video_id="vid1", video_ref=str(csv), tier="file",
85         baseline_path=str(tmp_path / "b.json"),

```

```

86         )
87         assert r.precision == 1.0 and r.recall == 1.0
88
89
90     # ----- CLI exit-code contract (CI gating) -----
91
92     def test_cli_update_baseline_then_clean_run(db, tmp_path, capsys, monkeypatch):
93         # Point the runner at our temp DB via --db; CSV ground truth via 'file' tier.
94         csv = tmp_path / "gt.csv"
95         csv.write_text("start,end\n0,10\n20,30\n")
96         base = str(tmp_path / "b.json")
97         db_path = db.db_path if hasattr(db, "db_path") else str(tmp_path / "t.db")
98
99         # First, snapshot the baseline ? exit 0.
100        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
101                                "--annotations", str(csv), "--db", db_path,
102                                "--baseline", base, "--update-baseline"])
103
104        assert rc == 0
105        assert "REGRESSED" not in capsys.readouterr().out
106
107    def test_cli_regression_exits_nonzero(tmp_path, capsys):
108        # DB missing one rally vs a baseline of perfect ? regression ? exit 1.
109        d = Database(str(tmp_path / "t.db"))
110        d.add_video("vid1", "/x/m.mp4", "processed", [])
111        d.add_play_segment("vid1", 0.0, 10.0)
112        csv = tmp_path / "gt.csv"
113        csv.write_text("start,end\n0,10\n20,30\n")
114        base = str(tmp_path / "b.json")
115        regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
116
117        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
118                                "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
119                                "--baseline", base])
120
121        assert rc == 1
122        assert "REGRESSED" in capsys.readouterr().out
123
124    def test_cli_missing_baseline_exits_3(db, tmp_path):
125        # No baseline + no --update/--allow ? the gate cannot verify ? exit 3 (not a silent
126        pass).
127        csv = tmp_path / "gt.csv"
128        csv.write_text("start,end\n0,10\n20,30\n")
129        db_path = str(tmp_path / "t.db")
130        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
131                                "--annotations", str(csv), "--db", db_path,
132                                "--baseline", str(tmp_path / "absent.json")])
133
134        assert rc == 3
135
136    def test_cli_missing_baseline_allowed_exits_0(db, tmp_path):
137        csv = tmp_path / "gt.csv"
138        csv.write_text("start,end\n0,10\n20,30\n")
139        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
140                                "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
141                                "--baseline", str(tmp_path / "absent.json"),
142                                "--allow-missing-baseline"])
143
144        assert rc == 0
145
146    def test_iou_mismatch_is_incommensurable(db, tmp_path):
147        p = str(tmp_path / "b.json")
148        regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
149                                gt_fingerprint=regression.gt_hash(GT))
150        # Same labels but scored at a different IoU ? not a valid comparison.

```

```

150         r = regression.regress(db, "vid1", GT, iou_threshold=0.3, baseline_path=p)
151         assert r.regressed is True
152         assert any("incommensurable" in why for why in r.reasons)
153
154
155     def test_changed_labels_detected_via_gt_hash(db, tmp_path):
156         p = str(tmp_path / "b.json")
157         regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
158                                 gt_fingerprint=regression.gt_hash(GT))
159         r = regression.regress(db, "vid1", [(0.0, 10.0), (21.0, 31.0)], iou_threshold=0.5,
160                                     baseline_path=p)
161         assert r.regressed is True
162         assert any("labels changed" in why for why in r.reasons)
163

```

12. assistant (2026-06-18T01:41:04.128Z)

Now I have a full picture. Let me assess the candidate against the REFUTE lens.

The key facts:

- ``b["precision"]`` and ``b["recall"]`` (line 129) are read with bare indexing, while sibling fields on lines 132 use ``b.get(...)``.
- Every record is written by ``save_baseline`` (line 80), which **always** writes both ``"precision"`` and ``"recall"`` keys (lines 92-95). There is no code path that produces a record lacking those keys.
- The committed ``regression_baseline.json`` doesn't even exist yet (only ``nightly_baseline.json``, ``heuristic_lovo_n6.json``, etc. ? none consumed by this ``regress()`` function). The file is a documented future artifact (M4, GS-5 in BACKLOG/GOLDEN_REGRESSION_FIXTURES).

So the *ONLY* trigger for this `KeyError` is a **hand-edited or externally-merged** baseline record that omits ``precision`/`recall`` while keeping the key. That is a hypothetical, out-of-band data-corruption scenario, not a trigger reachable by any code path or test in the repo.

Now the critical refutation point: even the candidate's own framing ("intended graceful degradation") doesn't hold up. The incommensurable-comparison guards (`iou/detector/gt_hash`) use ``get()`` because those fields are **legitimately optional** ? ``save_baseline``'s signature makes ``iou_threshold``, ``detector``, ``gt_fingerprint`` default to ``None``, so a record genuinely may lack a meaningful value. By contrast, ``precision`/`recall`/`f1`` are **required positional arguments** of ``save_baseline`` ? they are core, never-optional fields of the record contract. The bare indexing reflects an intentional invariant: a baseline record without precision/recall is malformed and there is no defined "graceful" behavior for it (what would a regression check even mean?). The asymmetry is by design, not an oversight.

There is no concrete, reachable trigger from a code path, no test, no committed data file, and the only path is hand-corruption of an internal artifact that is part of the trusted module's own contract. A maintainer would treat a precision-less baseline record as a corrupt file, where a crash is acceptable (and arguably preferable to silently treating it as "no baseline").

This is a defensive-coding nit, not a real defect.

13. user (2026-06-18T01:41:15.829Z)

Structured output provided successfully

14. assistant (2026-06-18T01:41:21.601Z)

Refuted. The candidate is not a genuine bug.

The bare ``b["precision"]`/`b["recall"]`` indexing at ``backend/eval/regression.py:129`` cannot be reached with a missing key by any code path: the sole writer, ``save_baseline`` (line 80), takes ``precision`/`recall`/`f1`` as required positional arguments and always writes them (lines 92-95). The ``get()`` asymmetry with line 132 is intentional ? ``iou_threshold`/`detector`/`gt_hash`` default to ``None`` in ``save_baseline`` and are legitimately optional, whereas precision/recall are the non-optional core of a baseline record with no defined graceful-degradation semantics. The

only "trigger" is hand-corruption of an internal trusted artifact that doesn't even exist in the worktree yet (a documented future M4/GS-5 file). A maintainer would treat a metric-less baseline entry as a corrupt file, not a defect.

real=false, reachable=false, intended_behavior=true.